Hands-on Assignment(Interacting with Oracle Database Server)

Write a program that displays the Netsal (Sal + Comm) of an employee entered at run time from EMP table. (Use NVL Function.)

Program:

```
SET SERVEROUTPUT ON;

DECLARE

   V_EMPNO EMP.EMPNO%TYPE;

   V_NETSAL NUMBER;

BEGIN

   V_EMPNO := &EMPNO;

 SELECT SAL + NVL(COMM, 0)

   INTO V_NETSAL

   FROM EMP

   WHERE EMPNO = V_EMPNO;

   DBMS_OUTPUT.PUT_LINE('Employee Number : ' || V_EMPNO);

   DBMS_OUTPUT.PUT_LINE('Net Salary      : ' || V_NETSAL);

 EXCEPTION

   WHEN NO_DATA_FOUND THEN

      DBMS_OUTPUT.PUT_LINE('Employee Not Found');

END;

/
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      V_EMPNO EMP.EMPNO%TYPE;
  3      V_NETSAL NUMBER;
  4  BEGIN
  5      V_EMPNO := &EMPNO;
  6  SELECT SAL + NVL(COMM, 0)
  7      INTO V_NETSAL
  8      FROM EMP
  9      WHERE EMPNO = V_EMPNO;
 10      DBMS_OUTPUT.PUT_LINE('Employee Number : ' || V_EMPNO);
 11      DBMS_OUTPUT.PUT_LINE('Net Salary      : ' || V_NETSAL);
 12  EXCEPTION
 13      WHEN NO_DATA_FOUND THEN
 14          DBMS_OUTPUT.PUT_LINE('Employee Not Found');
 15  END;
 16  /
Enter value for empno: 7499
old   5:      V_EMPNO := &EMPNO;
new   5:      V_EMPNO := 7499;
Employee Number : 7499
Net Salary      : 1900

PL/SQL procedure successfully completed.
```

Hands-on Assignment(Writing Control Structures)

1.Write a program that will prompt for a EMPNO, ENAME, SAL. If EMPNO does not exist in EMP table then INSERT the EMPNO, ENAME, SAL into the EMP table else UPDATE ENAME, SAL for that EMPNO.

Program:

```
SET SERVEROUTPUT ON
DECLARE
   V_EMPNO EMP.EMPNO%TYPE;
   V_ENAME EMP.ENAME%TYPE;
   V_SAL   EMP.SAL%TYPE;
   V_COUNT NUMBER;
BEGIN
   V_EMPNO := &EMPNO;
   V_ENAME := '&ENAME';
   V_SAL   := &SAL;
  SELECT COUNT(*)
   INTO V_COUNT
   FROM EMP
   WHERE EMPNO = V_EMPNO;
 IF V_COUNT = 0 THEN
     INSERT INTO EMP(EMPNO, ENAME, SAL)
      VALUES(V_EMPNO, V_ENAME, V_SAL);
       DBMS_OUTPUT.PUT_LINE('Record Inserted Successfully');
ELSE
     UPDATE EMP
      SET ENAME = V_ENAME,
         SAL = V_SAL
      WHERE EMPNO = V_EMPNO;
      DBMS_OUTPUT.PUT_LINE('Record Updated Successfully');
  END IF;
COMMIT;
END;
/
```

Output:

```
SQL> DECLARE
  2      V_EMPNO EMP.EMPNO%TYPE;
  3      V_ENAME EMP.ENAME%TYPE;
  4      V_SAL   EMP.SAL%TYPE;
  5      V_COUNT NUMBER;
  6  BEGIN
  7      V_EMPNO := &EMPNO;
  8      V_ENAME := '&ENAME';
  9      V_SAL   := &SAL;
 10   SELECT COUNT(*)
 11      INTO V_COUNT
 12      FROM EMP
 13      WHERE EMPNO = V_EMPNO;
 14   IF V_COUNT = 0 THEN
 15  INSERT INTO EMP(EMPNO, ENAME, SAL)
 16          VALUES(V_EMPNO, V_ENAME, V_SAL);
 17  DBMS_OUTPUT.PUT_LINE('Record Inserted Successfully');
 18  ELSE
 19   UPDATE EMP
 20          SET ENAME = V_ENAME,
 21              SAL = V_SAL
 22          WHERE EMPNO = V_EMPNO;
 23   DBMS_OUTPUT.PUT_LINE('Record Updated Successfully');
 24  END IF;
 25  COMMIT;
 26  END;
 27  /
Enter value for empno: 7369
old   7:     V_EMPNO := &EMPNO;
new   7:     V_EMPNO := 7369;
Enter value for ename: RAVI
old   8:     V_ENAME := '&ENAME';
new   8:     V_ENAME := 'RAVI';
Enter value for sal: 3000
old   9:     V_SAL   := &SAL;
new   9:     V_SAL   := 3000;
Record Updated Successfully

PL/SQL procedure successfully completed.
```

2. Write a program that will prompt for number and display whether the number is ODD or EVEN number.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   V_NUM NUMBER;
BEGIN
   V_NUM := &NUMBER;
   IF MOD(V_NUM, 2) = 0 THEN
      DBMS_OUTPUT.PUT_LINE(V_NUM || ' is EVEN number');
   ELSE
  DBMS_OUTPUT.PUT_LINE(V_NUM || ' is ODD number');
END IF;
END;
/
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      V_NUM NUMBER;
  3  BEGIN
  4      V_NUM := &NUMBER;
  5
  6      IF MOD(V_NUM, 2) = 0 THEN
  7          DBMS_OUTPUT.PUT_LINE(V_NUM || ' is EVEN number');
  8      ELSE
  9          DBMS_OUTPUT.PUT_LINE(V_NUM || ' is ODD number');
 10      END IF;
 11  END;
 12  /
Enter value for number: 20
old   4:     V_NUM := &NUMBER;
new   4:     V_NUM := 20;
20 is EVEN number

PL/SQL procedure successfully completed.
```

3.write program that prompts for EMPNO and update his salary based on the below specification:

If the employee belongs to DEPTNO 10 then update his salary by 10%.

If the employee belongs to DEPTNO 20 then update his salary by 15%.

If the employee belongs to other DEPTNO then update his salary with SALARY+COMM.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   V_EMPNO EMP.EMPNO%TYPE;
   V_SAL   EMP.SAL%TYPE;
   V_COMM  EMP.COMM%TYPE;
   V_DEPTNO EMP.DEPTNO%TYPE;
BEGIN
   V_EMPNO := &EMPNO;
  SELECT SAL, COMM, DEPTNO
   INTO V_SAL, V_COMM, V_DEPTNO
   FROM EMP
   WHERE EMPNO = V_EMPNO;
   IF V_DEPTNO = 10 THEN
      UPDATE EMP
      SET SAL = SAL + (SAL * 10 / 100)
      WHERE EMPNO = V_EMPNO;
      DBMS_OUTPUT.PUT_LINE('Salary updated by 10%');
   ELSIF V_DEPTNO = 20 THEN
      UPDATE EMP
      SET SAL = SAL + (SAL * 15 / 100)
      WHERE EMPNO = V_EMPNO;
      DBMS_OUTPUT.PUT_LINE('Salary updated by 15%');
    ELSE
      UPDATE EMP
      SET SAL = SAL + NVL(COMM, 0)
      WHERE EMPNO = V_EMPNO;
       DBMS_OUTPUT.PUT_LINE('Salary updated with SALARY + COMM');
  END IF;
```

```
  COMMIT;
END;
/
```

Output:

```
SQL> DECLARE
  2      V_EMPNO EMP.EMPNO%TYPE;
  3      V_SAL   EMP.SAL%TYPE;
  4      V_COMM  EMP.COMM%TYPE;
  5      V_DEPTNO EMP.DEPTNO%TYPE;
  6  BEGIN
  7      V_EMPNO := &EMPNO;
  8
  9      SELECT SAL, COMM, DEPTNO
 10      INTO V_SAL, V_COMM, V_DEPTNO
 11      FROM EMP
 12      WHERE EMPNO = V_EMPNO;
 13
 14      IF V_DEPTNO = 10 THEN
 15
 16          UPDATE EMP
 17          SET SAL = SAL + (SAL * 10 / 100)
 18          WHERE EMPNO = V_EMPNO;
 19
 20          DBMS_OUTPUT.PUT_LINE('Salary updated by 10%');
 21
 22      ELSIF V_DEPTNO = 20 THEN
 23
 24          UPDATE EMP
 25          SET SAL = SAL + (SAL * 15 / 100)
 26          WHERE EMPNO = V_EMPNO;
 27
 28          DBMS_OUTPUT.PUT_LINE('Salary updated by 15%');
 29
 30      ELSE
 31
 32          UPDATE EMP
 33          SET SAL = SAL + NVL(COMM, 0)
 34          WHERE EMPNO = V_EMPNO;
 35
 36          DBMS_OUTPUT.PUT_LINE('Salary updated with SALARY + COMM');
 37
 38      END IF;
 39
 40      COMMIT;
 41  END;
```

```
 42  /
Enter value for empno: 7369
old   7:     V_EMPNO := &EMPNO;
new   7:     V_EMPNO := 7369;
Salary updated by 15%

PL/SQL procedure successfully completed.
```

4.In sql Prompt creates a table MYTABLE1 with RESULT as a column of number datatype.

Write a program that will insert record in the mytable from 1,2,3… to 10 values.

Do not insert record 6 and 8

Program:

```
CREATE TABLE MYTABLE1 (RESULT NUMBER);
SET SERVEROUTPUT ON;
BEGIN
   FOR I IN 1..10
   LOOP
      IF I <> 6 AND I <> 8 THEN
        INSERT INTO MYTABLE1(RESULT)
         VALUES(I);
       END IF;
   END LOOP;
COMMIT;
  DBMS_OUTPUT.PUT_LINE('Records Inserted Successfully');
END;
/
SELECT * FROM MYTABLE1;
```

Output:

```
SQL> CREATE TABLE MYTABLE1
  2  (
  3      RESULT NUMBER
  4  );

Table created.

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
  2      FOR I IN 1..10
  3      LOOP
  4          IF I <> 6 AND I <> 8 THEN
  5
  6              INSERT INTO MYTABLE1(RESULT)
  7              VALUES(I);
  8
  9          END IF;
 10      END LOOP;
 11
 12      COMMIT;
 13
 14      DBMS_OUTPUT.PUT_LINE('Records Inserted Successfully');
 15  END;
 16  /
Records Inserted Successfully

PL/SQL procedure successfully completed.
```

```
SQL> SELECT * FROM MYTABLE1;

    RESULT
----------
         1
         2
         3
         4
         5
         7
         9
        10

8 rows selected.
```

Hands-on Assignment(Working With Composite Data Types)

Write a program that will display the entire row by passing the EMPNO. (Use %ROWTYPE)

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   V_EMPNO EMP.EMPNO%TYPE;
   V_EMP EMP%ROWTYPE;
BEGIN
   V_EMPNO := &EMPNO;
   SELECT *
   INTO V_EMP
   FROM EMP
   WHERE EMPNO = V_EMPNO;
   DBMS_OUTPUT.PUT_LINE('EMPNO  : ' || V_EMP.EMPNO);
   DBMS_OUTPUT.PUT_LINE('ENAME  : ' || V_EMP.ENAME);
   DBMS_OUTPUT.PUT_LINE('JOB    : ' || V_EMP.JOB);
   DBMS_OUTPUT.PUT_LINE('SAL    : ' || V_EMP.SAL);
   DBMS_OUTPUT.PUT_LINE('COMM   : ' || V_EMP.COMM);
   DBMS_OUTPUT.PUT_LINE('DEPTNO : ' || V_EMP.DEPTNO);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Employee Not Found');
END;
/
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      V_EMPNO EMP.EMPNO%TYPE;
  3      V_EMP EMP%ROWTYPE;
  4  BEGIN
  5      V_EMPNO := &EMPNO;
  6
  7      SELECT *
  8      INTO V_EMP
  9      FROM EMP
 10      WHERE EMPNO = V_EMPNO;
 11
 12      DBMS_OUTPUT.PUT_LINE('EMPNO  : ' || V_EMP.EMPNO);
 13      DBMS_OUTPUT.PUT_LINE('ENAME  : ' || V_EMP.ENAME);
 14      DBMS_OUTPUT.PUT_LINE('JOB    : ' || V_EMP.JOB);
 15      DBMS_OUTPUT.PUT_LINE('SAL    : ' || V_EMP.SAL);
 16      DBMS_OUTPUT.PUT_LINE('COMM   : ' || V_EMP.COMM);
 17      DBMS_OUTPUT.PUT_LINE('DEPTNO : ' || V_EMP.DEPTNO);
 18
 19  EXCEPTION
 20      WHEN NO_DATA_FOUND THEN
 21          DBMS_OUTPUT.PUT_LINE('Employee Not Found');
 22  END;
 23  /
Enter value for empno: 7499
old   5:     V_EMPNO := &EMPNO;
new   5:     V_EMPNO := 7499;
EMPNO  : 7499
ENAME  : ALLEN
JOB    : SALESMAN
SAL    : 1600
COMM   : 300
DEPTNO : 30

PL/SQL procedure successfully completed.
```

Hands-on Assignment(Explicit Cursor)

1. Write a implicit cursor that will prompt for a JOB and delete the employees working in that Job. Store the number of records deleted in a session variable.

Program:

```
SET SERVEROUTPUT ON;
VARIABLE V_COUNT NUMBER;
DECLARE
   V_JOB EMP.JOB%TYPE;
BEGIN
   V_JOB := '&JOB';
   DELETE FROM EMP
   WHERE JOB = V_JOB;
   :V_COUNT := SQL%ROWCOUNT;
   DBMS_OUTPUT.PUT_LINE('Number of records deleted: ' || :V_COUNT);
   COMMIT;
END;
/
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> VARIABLE V_COUNT NUMBER;
SQL>
SQL> DECLARE
  2      V_JOB EMP.JOB%TYPE;
  3  BEGIN
  4      V_JOB := '&JOB';
  5
  6      DELETE FROM EMP
  7      WHERE JOB = V_JOB;
  8
  9      :V_COUNT := SQL%ROWCOUNT;
 10
 11      DBMS_OUTPUT.PUT_LINE('Number of records deleted: ' || :V_COUNT);
 12
 13      COMMIT;
 14  END;
 15  /
Enter value for job: CLERK
old   4:     V_JOB := '&JOB';
new   4:     V_JOB := 'CLERK';
Number of records deleted: 4

PL/SQL procedure successfully completed.
```

2.Rollback the previous Delete Statement.

Query:

ROLLBACK;

SELECT * FROM EMP;

Output:

```
SQL> ROLLBACK;

Rollback complete.

SQL> SELECT * FROM EMP;

     EMPNO ENAME                JOB                  MGR HIREDATE
---------- -------------------- ------------------ ---------- ---------
       SAL       COMM     DEPTNO
---------- ---------- ----------
      7499 ALLEN                SALESMAN                7698 20-FEB-81
      1600        300         30

      7521 WARD                 SALESMAN                7698 22-FEB-81
      1250        500         30

      7566 JONES                MANAGER                 7839 02-APR-81
      2975                    20


     EMPNO ENAME                JOB                  MGR HIREDATE
---------- -------------------- ------------------ ---------- ---------
       SAL       COMM     DEPTNO
---------- ---------- ----------
      7654 MARTIN               SALESMAN                7698 28-SEP-81
      1250       1400         30

      7698 BLAKE                MANAGER                 7839 01-MAY-81
      2850                    30

      7788 SCOTT                ANALYST                 7566 19-APR-87
      3000                    20


     EMPNO ENAME                JOB                  MGR HIREDATE
---------- -------------------- ------------------ ---------- ---------
       SAL       COMM     DEPTNO
---------- ---------- ----------
      7782 CLARK                MANAGER                 7839 09-JUN-81
      2450                    10
```

```
      7839 KING                 PRESIDENT                    17-NOV-81
      5000                    10

      7844 TURNER               SALESMAN                7698 08-SEP-81
      1500          0         30


     EMPNO ENAME                JOB                  MGR HIREDATE
---------- -------------------- ------------------ ---------- ---------
       SAL       COMM     DEPTNO
---------- ---------- ----------
      7902 FORD                 ANALYST                 7566 03-DEC-81
      3000                    20


10 rows selected.
```

3.Write a program that displays the Top N salaries using simple Explicit Cursors.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
    CURSOR C_EMP(V_N NUMBER) IS
        SELECT ENAME, SAL
        FROM EMP
        ORDER BY SAL DESC;
    V_NAME EMP.ENAME%TYPE;
    V_SAL  EMP.SAL%TYPE;
    V_N NUMBER;
    V_COUNT NUMBER := 0;
BEGIN
    V_N := &N;
 OPEN C_EMP(V_N);
  LOOP
        FETCH C_EMP INTO V_NAME, V_SAL;
        EXIT WHEN C_EMP%NOTFOUND OR V_COUNT = V_N;
        V_COUNT := V_COUNT + 1;
       DBMS_OUTPUT.PUT_LINE(V_COUNT || '. ' || V_NAME || ' - ' || V_SAL);
   END LOOP;
  CLOSE C_EMP;
END;
/
```

Output:

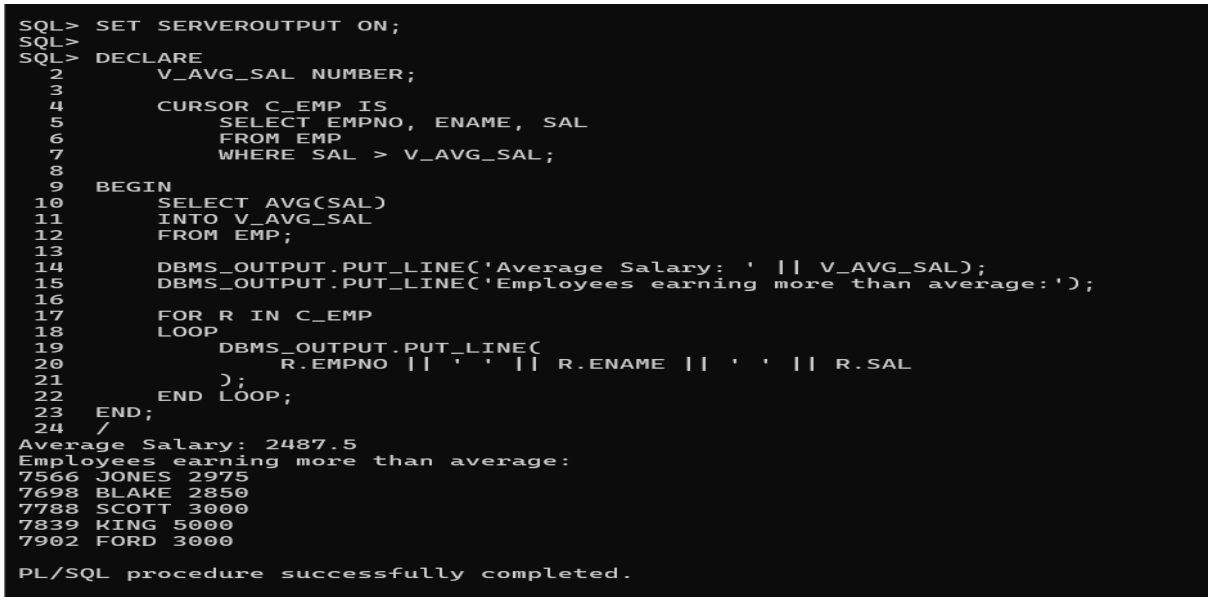
```
SQL> DECLARE
  2      CURSOR C_EMP(V_N NUMBER) IS
  3          SELECT ENAME, SAL
  4          FROM EMP
  5          ORDER BY SAL DESC;
  6
  7      V_NAME EMP.ENAME%TYPE;
  8      V_SAL  EMP.SAL%TYPE;
  9      V_N NUMBER;
 10      V_COUNT NUMBER := 0;
 11  BEGIN
 12      V_N := &N;
 13
 14      OPEN C_EMP(V_N);
 15
 16      LOOP
 17          FETCH C_EMP INTO V_NAME, V_SAL;
 18          EXIT WHEN C_EMP%NOTFOUND OR V_COUNT = V_N;
 19
 20          V_COUNT := V_COUNT + 1;
 21
 22          DBMS_OUTPUT.PUT_LINE(
 23              V_COUNT || '. ' || V_NAME || ' - ' || V_SAL
 24          );
 25      END LOOP;
 26
 27      CLOSE C_EMP;
 28  END;
 29  /
Enter value for n: 5
old  12:     V_N := &N;
new  12:     V_N := 5;
1. KING - 5000
2. SCOTT - 3000
3. FORD - 3000
4. JONES - 2975
5. BLAKE - 2850

PL/SQL procedure successfully completed.
```

4.Create a Simple Explicit Cursor that displays all the employees who are earning more than the average salary of the EMP table.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   V_AVG_SAL NUMBER;
 CURSOR C_EMP IS
      SELECT EMPNO, ENAME, SAL
      FROM EMP
      WHERE SAL > V_AVG_SAL;
BEGIN
   SELECT AVG(SAL)
   INTO V_AVG_SAL
   FROM EMP;
    DBMS_OUTPUT.PUT_LINE('Average Salary: ' || V_AVG_SAL);
   DBMS_OUTPUT.PUT_LINE('Employees earning more than average:');
  FOR R IN C_EMP
   LOOP
      DBMS_OUTPUT.PUT_LINE(R.EMPNO || ' ' || R.ENAME || ' ' || R.SAL);
   END LOOP;
END;
/
```

Output:



```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2       V_AVG_SAL NUMBER;
  3
  4       CURSOR C_EMP IS
  5           SELECT EMPNO, ENAME, SAL
  6           FROM EMP
  7           WHERE SAL > V_AVG_SAL;
  8
  9  BEGIN
 10       SELECT AVG(SAL)
 11       INTO V_AVG_SAL
 12       FROM EMP;
 13
 14       DBMS_OUTPUT.PUT_LINE('Average Salary: ' || V_AVG_SAL);
 15       DBMS_OUTPUT.PUT_LINE('Employees earning more than average:');
 16
 17       FOR R IN C_EMP
 18       LOOP
 19           DBMS_OUTPUT.PUT_LINE(
 20               R.EMPNO || ' ' || R.ENAME || ' ' || R.SAL
 21           );
 22       END LOOP;
 23  END;
 24  /
Average Salary: 2487.5
Employees earning more than average:
7566 JONES 2975
7698 BLAKE 2850
7788 SCOTT 3000
7839 KING 5000
7902 FORD 3000

PL/SQL procedure successfully completed.
```

5. Create a Simple Explicit cursor that displays the employees who earn more than 2000 and have joined the company after 15-Jun-1981.
Print the output: < > earns < > and joined the organization on <>.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR C_EMP IS
      SELECT ENAME, SAL, HIREDATE
      FROM EMP
      WHERE SAL > 2000
      AND HIREDATE > TO_DATE('15-JUN-1981','DD-MON-YYYY');
BEGIN
   FOR R IN C_EMP
   LOOP
      DBMS_OUTPUT.PUT_LINE(
         R.ENAME || ' earns ' || R.SAL ||
         ' and joined the organization on ' || TO_CHAR(R.HIREDATE, 'DD-MON-YYYY'));

   END LOOP;

   END;
   /
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      CURSOR C_EMP IS
  3          SELECT ENAME, SAL, HIREDATE
  4          FROM EMP
  5          WHERE SAL > 2000
  6          AND HIREDATE > TO_DATE('15-JUN-1981','DD-MON-YYYY');
  7
  8  BEGIN
  9      FOR R IN C_EMP
 10      LOOP
 11          DBMS_OUTPUT.PUT_LINE(
 12              R.ENAME || ' earns ' || R.SAL ||
 13              ' and joined the organization on ' ||
 14              TO_CHAR(R.HIREDATE, 'DD-MON-YYYY')
 15          );
 16      END LOOP;
 17  END;
 18  /
SCOTT earns 3000 and joined the organization on 19-APR-1987
KING earns 5000 and joined the organization on 17-NOV-1981
FORD earns 3000 and joined the organization on 03-DEC-1981

PL/SQL procedure successfully completed.
```

6. Create a Simple Explicit cursor that displays the hiredate in the format **DD-MM-RRRR** and Day. Sort the records on DAY starting from Saturday.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR C_EMP IS
      SELECT ENAME,
           HIREDATE,
           TO_CHAR(HIREDATE, 'DD-MM-RRRR') AS HDATE,
           TO_CHAR(HIREDATE, 'DAY') AS HDAY,
           TO_CHAR(HIREDATE, 'D') AS DAY_NO
      FROM EMP  ORDER BY  MOD(TO_NUMBER(TO_CHAR(HIREDATE, 'D')) + 1, 7);
BEGIN
   FOR R IN C_EMP
   LOOP
      DBMS_OUTPUT.PUT_LINE(   R.ENAME || ' - ' ||R.HDATE || ' - ' || TRIM(R.HDAY) );
   END LOOP;
END;
/
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      CURSOR C_EMP IS
  3          SELECT ENAME,
  4                 HIREDATE,
  5                 TO_CHAR(HIREDATE, 'DD-MM-RRRR') AS HDATE,
  6                 TO_CHAR(HIREDATE, 'DAY') AS HDAY,
  7                 TO_CHAR(HIREDATE, 'D') AS DAY_NO
  8          FROM EMP
  9          ORDER BY
 10              MOD(TO_NUMBER(TO_CHAR(HIREDATE, 'D')) + 1, 7);
 11
 12  BEGIN
 13      FOR R IN C_EMP
 14      LOOP
 15          DBMS_OUTPUT.PUT_LINE(
 16              R.ENAME || ' - ' ||
 17              R.HDATE || ' - ' ||
 18              TRIM(R.HDAY)
 19          );
 20      END LOOP;
 21  END;
 22  /
ALLEN - 20-02-1981 - FRIDAY
BLAKE - 01-05-1981 - FRIDAY
SCOTT - 19-04-1987 - SUNDAY
WARD - 22-02-1981 - SUNDAY
MARTIN - 28-09-1981 - MONDAY
CLARK - 09-06-1981 - TUESDAY
KING - 17-11-1981 - TUESDAY
TURNER - 08-09-1981 - TUESDAY
JONES - 02-04-1981 - THURSDAY
FORD - 03-12-1981 - THURSDAY

PL/SQL procedure successfully completed.
```

7. Create a simple explicit cursor that displays the employees names and commission amounts. If an employee does not earn commission, show **"No Commission"**. Label the column COMM.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR C_EMP IS
      SELECT ENAME,
                NVL(TO_CHAR(COMM), 'No Commission') AS COMM FROM EMP;
BEGIN
   DBMS_OUTPUT.PUT_LINE(RPAD('ENAME', 15) || 'COMM' );
DBMS_OUTPUT.PUT_LINE('----------------------------' );

  FOR R IN C_EMP

   LOOP

    DBMS_OUTPUT.PUT_LINE(RPAD(R.ENAME, 15) || R.COMM  );

   END LOOP;
END;
/
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      CURSOR C_EMP IS
  3          SELECT ENAME,
  4                 NVL(TO_CHAR(COMM), 'No Commission') AS COMM
  5          FROM EMP;
  6
  7  BEGIN
  8      DBMS_OUTPUT.PUT_LINE(
  9          RPAD('ENAME', 15) || 'COMM'
 10      );
 11
 12      DBMS_OUTPUT.PUT_LINE(
 13          '----------------------------'
 14      );
 15
 16      FOR R IN C_EMP
 17      LOOP
 18          DBMS_OUTPUT.PUT_LINE(
 19              RPAD(R.ENAME, 15) || R.COMM
 20          );
 21      END LOOP;
 22  END;
 23  /
ENAME          COMM
----------------------------
ALLEN          300
WARD           500
JONES          No Commission
MARTIN         1400
BLAKE          No Commission
SCOTT          No Commission
CLARK          No Commission
KING           No Commission
TURNER         0
FORD           No Commission

PL/SQL procedure successfully completed.
```

8. Create a parameter cursor that takes DEPTNO as a parameter and update the star column with *. Every * represents a 1000.
Example: If the employee salary is 5000 then update star column with *****.
Use Parameter Cursor, Cursor with FOR LOOP and WHERE CURRENT OF option.
Program:

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR C_EMP(P_DEPTNO EMP.DEPTNO%TYPE) IS
      SELECT SAL, STAR
      FROM EMP
      WHERE DEPTNO = P_DEPTNO
      FOR UPDATE OF STAR;
    V_DEPTNO NUMBER;
BEGIN
   V_DEPTNO := &DEPTNO;
 FOR R IN C_EMP(V_DEPTNO)
   LOOP
      UPDATE EMP
      SET STAR = RPAD('*', TRUNC(R.SAL / 1000), '*')
      WHERE CURRENT OF C_EMP;
   END LOOP;

 COMMIT;

    DBMS_OUTPUT.PUT_LINE('STAR column updated successfully.');

END;
/
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      CURSOR C_EMP(P_DEPTNO EMP.DEPTNO%TYPE) IS
  3          SELECT SAL, STAR
  4          FROM EMP
  5          WHERE DEPTNO = P_DEPTNO
  6          FOR UPDATE OF STAR;
  7
  8      V_DEPTNO NUMBER;
  9  BEGIN
 10      V_DEPTNO := &DEPTNO;
 11
 12      FOR R IN C_EMP(V_DEPTNO)
 13      LOOP
 14          UPDATE EMP
 15          SET STAR = RPAD('*', TRUNC(R.SAL / 1000), '*')
 16          WHERE CURRENT OF C_EMP;
 17      END LOOP;
 18
 19      COMMIT;
 20
 21      DBMS_OUTPUT.PUT_LINE('STAR column updated successfully.');
 22  END;
 23  /
Enter value for deptno: 20
old  10:      V_DEPTNO := &DEPTNO;
new  10:      V_DEPTNO := 20;
STAR column updated successfully.

PL/SQL procedure successfully completed.
```

9. Write a Parameter Cursor program that promotes **CLERK** who earn more than **1000** to **SR CLERK** and increase the salary by **10%**. Pass CLERK as a parameter to the Cursor. Use Parameter Cursor, Cursor with FOR LOOP and Cursor with UPDATE clause.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR C_EMP(P_JOB EMP.JOB%TYPE) IS
      SELECT ENAME, SAL
      FROM EMP
      WHERE JOB = P_JOB
      AND SAL > 1000
      FOR UPDATE OF JOB, SAL;
BEGIN
   FOR R IN C_EMP('CLERK')
   LOOP
      UPDATE EMP
      SET JOB = 'SR CLERK',
         SAL = SAL + (SAL * 10 / 100)
      WHERE CURRENT OF C_EMP;
     DBMS_OUTPUT.PUT_LINE(R.ENAME || ' promoted to SR CLERK' );
   END LOOP;
COMMIT;
END;
/
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      CURSOR C_EMP(P_JOB EMP.JOB%TYPE) IS
  3          SELECT ENAME, SAL
  4          FROM EMP
  5          WHERE JOB = P_JOB
  6          AND SAL > 1000
  7          FOR UPDATE OF JOB, SAL;
  8
  9  BEGIN
 10      FOR R IN C_EMP('CLERK')
 11      LOOP
 12          UPDATE EMP
 13          SET JOB = 'SR CLERK',
 14              SAL = SAL + (SAL * 10 / 100)
 15          WHERE CURRENT OF C_EMP;
 16
 17          DBMS_OUTPUT.PUT_LINE(
 18              R.ENAME || ' promoted to SR CLERK'
 19          );
 20      END LOOP;
 21
 22      COMMIT;
 23  END;
 24  /

PL/SQL procedure successfully completed.
```

10. Display entire record from the DEPT table by passing a DEPTNO. Increase the Salary of employees working in deptno 10 by 15%, deptno 20 by 15% and others by 5%. Also display the corresponding employees working in that Dept.
Use a parameter Cursor, For Loop Cursor and Cursor with Update clause.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   V_DEPTNO DEPT.DEPTNO%TYPE;
   V_DNAME DEPT.DNAME%TYPE;
   V_LOC   DEPT.LOC%TYPE;
   CURSOR C_EMP(P_DEPTNO EMP.DEPTNO%TYPE) IS
      SELECT EMPNO, ENAME, SAL
      FROM EMP
      WHERE DEPTNO = P_DEPTNO
      FOR UPDATE OF SAL;
BEGIN
   V_DEPTNO := &DEPTNO;
   SELECT DNAME, LOC
   INTO V_DNAME, V_LOC
   FROM DEPT
   WHERE DEPTNO = V_DEPTNO;
   DBMS_OUTPUT.PUT_LINE('DEPTNO : ' || V_DEPTNO);
   DBMS_OUTPUT.PUT_LINE('DNAME  : ' || V_DNAME);
   DBMS_OUTPUT.PUT_LINE('LOC    : ' || V_LOC);
   DBMS_OUTPUT.PUT_LINE('Employees:');
  FOR R IN C_EMP(V_DEPTNO)
   LOOP
      DBMS_OUTPUT.PUT_LINE(R.EMPNO || ' ' || R.ENAME ||' Salary: ' || R.SAL );
    IF V_DEPTNO = 10 THEN
        UPDATE EMP
        SET SAL = SAL + (SAL * 15 / 100)
        WHERE CURRENT OF C_EMP;
    ELSIF V_DEPTNO = 20 THEN
       UPDATE EMP
        SET SAL = SAL + (SAL * 15 / 100)
```

```
        WHERE CURRENT OF C_EMP;
  ELSE
        UPDATE EMP
        SET SAL = SAL + (SAL * 5 / 100)
        WHERE CURRENT OF C_EMP;
      END IF;
   END LOOP;
COMMIT;
END;
/
```

Output:

```
SQL> DECLARE
  2      V_DEPTNO DEPT.DEPTNO%TYPE;
  3
  4      V_DNAME DEPT.DNAME%TYPE;
  5      V_LOC   DEPT.LOC%TYPE;
  6
  7      CURSOR C_EMP(P_DEPTNO EMP.DEPTNO%TYPE) IS
  8          SELECT EMPNO, ENAME, SAL
  9          FROM EMP
 10          WHERE DEPTNO = P_DEPTNO
 11          FOR UPDATE OF SAL;
 12
 13  BEGIN
 14      V_DEPTNO := &DEPTNO;
 15
 16      SELECT DNAME, LOC
 17      INTO V_DNAME, V_LOC
 18      FROM DEPT
 19      WHERE DEPTNO = V_DEPTNO;
 20
 21      DBMS_OUTPUT.PUT_LINE('DEPTNO : ' || V_DEPTNO);
 22      DBMS_OUTPUT.PUT_LINE('DNAME  : ' || V_DNAME);
 23      DBMS_OUTPUT.PUT_LINE('LOC    : ' || V_LOC);
 24
 25      DBMS_OUTPUT.PUT_LINE('Employees:');
 26
 27      FOR R IN C_EMP(V_DEPTNO)
 28      LOOP
 29          DBMS_OUTPUT.PUT_LINE(
 30              R.EMPNO || ' ' || R.ENAME ||
 31              ' Salary: ' || R.SAL
 32          );
 33
 34          IF V_DEPTNO = 10 THEN
 35              UPDATE EMP
 36              SET SAL = SAL + (SAL * 15 / 100)
 37              WHERE CURRENT OF C_EMP;
 38
 39          ELSIF V_DEPTNO = 20 THEN
 40              UPDATE EMP
```

```
 41              SET SAL = SAL + (SAL * 15 / 100)
 42              WHERE CURRENT OF C_EMP;
 43
 44          ELSE
 45              UPDATE EMP
 46              SET SAL = SAL + (SAL * 5 / 100)
 47              WHERE CURRENT OF C_EMP;
 48          END IF;
 49      END LOOP;
 50
 51      COMMIT;
 52  END;
 53  /
Enter value for deptno: 10
old  14:     V_DEPTNO := &DEPTNO;
new  14:     V_DEPTNO := 10;
DEPTNO : 10
DNAME  : ACCOUNTING
LOC    : NEW YORK
Employees:
7782 CLARK Salary: 2450
7839 KING Salary: 5000

PL/SQL procedure successfully completed.
```

11. Write a program that will prompt the user for a **CHOICE**. If CHOICE is 1 then display all the employees who earn more than 2000 from EMP table else display all the Employees who earn less than 2000.Use **REF CURSORS**.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   TYPE REF_CURSOR_TYPE IS REF CURSOR;
   C_EMP REF_CURSOR_TYPE;
   V_CHOICE NUMBER;
   V_EMPNO EMP.EMPNO%TYPE;
   V_ENAME EMP.ENAME%TYPE;
   V_SAL   EMP.SAL%TYPE;
BEGIN
   V_CHOICE := &CHOICE;
IF V_CHOICE = 1 THEN
    OPEN C_EMP FOR
         SELECT EMPNO, ENAME, SAL
         FROM EMP WHERE SAL > 2000;
 DBMS_OUTPUT.PUT_LINE('Employees earning more than 2000:' );
 ELSE
  OPEN C_EMP FOR
         SELECT EMPNO, ENAME, SAL
         FROM EMP
         WHERE SAL < 2000;
     DBMS_OUTPUT.PUT_LINE('Employees earning less than 2000:');
 END IF;
LOOP
      FETCH C_EMP INTO V_EMPNO, V_ENAME, V_SAL;
      EXIT WHEN C_EMP%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE( V_EMPNO || ' ' || V_ENAME || ' ' || V_SAL   );
END LOOP;
CLOSE C_EMP;
END;
/
```

Output:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      TYPE REF_CURSOR_TYPE IS REF CURSOR;
  3
  4      C_EMP REF_CURSOR_TYPE;
  5
  6      V_CHOICE NUMBER;
  7
  8      V_EMPNO EMP.EMPNO%TYPE;
  9      V_ENAME EMP.ENAME%TYPE;
 10      V_SAL   EMP.SAL%TYPE;
 11
 12  BEGIN
 13      V_CHOICE := &CHOICE;
 14
 15      IF V_CHOICE = 1 THEN
 16
 17          OPEN C_EMP FOR
 18              SELECT EMPNO, ENAME, SAL
 19              FROM EMP
 20              WHERE SAL > 2000;
 21
 22          DBMS_OUTPUT.PUT_LINE(
 23              'Employees earning more than 2000:'
 24          );
 25
 26      ELSE
 27
 28          OPEN C_EMP FOR
 29              SELECT EMPNO, ENAME, SAL
 30              FROM EMP
 31              WHERE SAL < 2000;
 32
 33          DBMS_OUTPUT.PUT_LINE(
 34              'Employees earning less than 2000:'
 35          );
 36
 37      END IF;
 38
```

```
 39      LOOP
 40          FETCH C_EMP INTO V_EMPNO, V_ENAME, V_SAL;
 41
 42          EXIT WHEN C_EMP%NOTFOUND;
 43
 44          DBMS_OUTPUT.PUT_LINE(
 45              V_EMPNO || ' ' || V_ENAME || ' ' || V_SAL
 46          );
 47      END LOOP;
 48
 49      CLOSE C_EMP;
 50  END;
 51  /
Enter value for choice: 1
old  13:     V_CHOICE := &CHOICE;
new  13:     V_CHOICE := 1;
Employees earning more than 2000:
7566 JONES 2975
7698 BLAKE 2850
7788 SCOTT 3000
7782 CLARK 2817.5
7839 KING 5750
7902 FORD 3000

PL/SQL procedure successfully completed.
```

Hands-on Assignment(Handling Exceptions)

1. Create a table `MESSAGES` with a column `RESULT` of type `VARCHAR2(1000)`.

Write a program that prompts for `SAL` and gets the corresponding `ENAME` from the `EMP` table. Do not use Explicit Cursors.

If the value entered returns one row, insert into `MESSAGES` as `ENAME || '|' || SAL`.

If the value returned is more than one row, insert into `MESSAGES` as `'More than one employee with a salary <>'`.

If no value is returned, insert into `MESSAGES` as `'No Employee with that salary <>'`.

Any other error insert `'Other Error'`.

Use all NAMED EXCEPTION HANDLERS.

Program:

```
CREATE TABLE MESSAGES (RESULT VARCHAR2(1000));

SET SERVEROUTPUT ON;

DECLARE

   V_SAL   EMP.SAL%TYPE;

   V_ENAME EMP.ENAME%TYPE;

BEGIN

   V_SAL := &SAL;

   SELECT ENAME  INTO V_ENAME FROM EMP

   WHERE SAL = V_SAL;

   INSERT INTO MESSAGES

   VALUES (V_ENAME || '|' || V_SAL);

   DBMS_OUTPUT.PUT_LINE('Record inserted successfully.');

EXCEPTION

 WHEN TOO_MANY_ROWS THEN

 INSERT INTO MESSAGES

       VALUES ('More than one employee with a salary ' || V_SAL);
```

```
    DBMS_OUTPUT.PUT_LINE('More than one employee with a salary ' || V_SAL );
  WHEN NO_DATA_FOUND THEN
      INSERT INTO MESSAGES
      VALUES ('No Employee with that salary ' || V_SAL);
       DBMS_OUTPUT.PUT_LINE('No Employee with that salary ' || V_SAL);
  WHEN OTHERS THEN
     INSERT INTO MESSAGES
      VALUES ('Other Error');
      DBMS_OUTPUT.PUT_LINE('Other Error');
END;
/
```

Output:

```
SQL> CREATE TABLE MESSAGES
  2  (
  3      RESULT VARCHAR2(1000)
  4  );

Table created.

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      V_SAL   EMP.SAL%TYPE;
  3      V_ENAME EMP.ENAME%TYPE;
  4  BEGIN
  5      V_SAL := &SAL;
  6
  7      SELECT ENAME
  8      INTO V_ENAME
  9      FROM EMP
 10      WHERE SAL = V_SAL;
 11
 12      INSERT INTO MESSAGES
 13      VALUES (V_ENAME || '|' || V_SAL);
 14
 15      DBMS_OUTPUT.PUT_LINE('Record inserted successfully.');
 16
 17  EXCEPTION
 18
 19      WHEN TOO_MANY_ROWS THEN
 20
 21          INSERT INTO MESSAGES
 22          VALUES ('More than one employee with a salary ' || V_SAL);
 23
 24          DBMS_OUTPUT.PUT_LINE(
 25              'More than one employee with a salary ' || V_SAL
 26          );
 27
 28      WHEN NO_DATA_FOUND THEN
 29
 30          INSERT INTO MESSAGES
 31          VALUES ('No Employee with that salary ' || V_SAL);
```

```
 32
 33          DBMS_OUTPUT.PUT_LINE(
 34              'No Employee with that salary ' || V_SAL
 35          );
 36
 37      WHEN OTHERS THEN
 38
 39          INSERT INTO MESSAGES
 40          VALUES ('Other Error');
 41
 42          DBMS_OUTPUT.PUT_LINE('Other Error');
 43
 44  END;
 45  /
Enter value for sal: 800
old   5:      V_SAL := &SAL;
new   5:      V_SAL := 800;
No Employee with that salary 800

PL/SQL procedure successfully completed.

SQL> SELECT * FROM MESSAGES;

RESULT
--------------------------------------------------------------------------------
----
No Employee with that salary 800
```

2. Write a program that will prompt for EMPNO, ENAME, SAL and insert them into the EMP table.

Use Unnamed system-defined exception handler.

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   V_EMPNO EMP.EMPNO%TYPE;
   V_ENAME EMP.ENAME%TYPE;
   V_SAL   EMP.SAL%TYPE;
E_DUPLICATE EXCEPTION;
PRAGMA EXCEPTION_INIT(E_DUPLICATE, -1);
BEGIN
   V_EMPNO := &EMPNO;
   V_ENAME := '&ENAME';
   V_SAL   := &SAL;
   INSERT INTO EMP(EMPNO, ENAME, SAL)
   VALUES(V_EMPNO, V_ENAME, V_SAL);
COMMIT;
DBMS_OUTPUT.PUT_LINE('Employee inserted successfully.');
EXCEPTION
 WHEN E_DUPLICATE THEN
       DBMS_OUTPUT.PUT_LINE( 'Employee number already exists.');
 WHEN OTHERS THEN
       DBMS_OUTPUT.PUT_LINE('Other Error: ' || SQLERRM);
END;
/
```

Output:

```
SQL> DECLARE
  2      V_EMPNO EMP.EMPNO%TYPE;
  3      V_ENAME EMP.ENAME%TYPE;
  4      V_SAL   EMP.SAL%TYPE;
  5
  6      E_DUPLICATE EXCEPTION;
  7
  8      PRAGMA EXCEPTION_INIT(E_DUPLICATE, -1);
  9
 10  BEGIN
 11      V_EMPNO := &EMPNO;
 12      V_ENAME := '&ENAME';
 13      V_SAL   := &SAL;
 14
 15      INSERT INTO EMP(EMPNO, ENAME, SAL)
 16      VALUES(V_EMPNO, V_ENAME, V_SAL);
 17
 18      COMMIT;
 19
 20      DBMS_OUTPUT.PUT_LINE('Employee inserted successfully.');
 21
 22  EXCEPTION
 23
 24      WHEN E_DUPLICATE THEN
 25          DBMS_OUTPUT.PUT_LINE(
 26              'Employee number already exists.'
 27          );
 28
 29      WHEN OTHERS THEN
 30          DBMS_OUTPUT.PUT_LINE(
 31              'Other Error: ' || SQLERRM
 32          );
 33
 34  END;
 35  /
```

```
Enter value for empno: 7369
old  11:     V_EMPNO := &EMPNO;
new  11:     V_EMPNO := 7369;
Enter value for ename: TEST
old  12:     V_ENAME := '&ENAME';
new  12:     V_ENAME := 'TEST';
Enter value for sal: 3000
old  13:     V_SAL   := &SAL;
new  13:     V_SAL   := 3000;
Employee inserted successfully.

PL/SQL procedure successfully completed.
```

3.Write a program that will prompt for an EMPNO and delete the employee from the EMP table. Handle with User Defined Exceptions**.**

Program:

```
SET SERVEROUTPUT ON;

DECLARE

   V_EMPNO EMP.EMPNO%TYPE;

   E_EMP_NOT_FOUND EXCEPTION;

BEGIN

   V_EMPNO := &EMPNO;

   DELETE FROM EMP  WHERE EMPNO = V_EMPNO;

   IF SQL%ROWCOUNT = 0 THEN

      RAISE E_EMP_NOT_FOUND;

   END IF;

 COMMIT;

DBMS_OUTPUT.PUT_LINE('Employee ' || V_EMPNO || ' deleted successfully.' );

EXCEPTION

WHEN E_EMP_NOT_FOUND THEN

      DBMS_OUTPUT.PUT_LINE('Employee ' || V_EMPNO || ' not found.' );

WHEN OTHERS THEN

      DBMS_OUTPUT.PUT_LINE('Other Error: ' || SQLERRM );

END;

/
```

Output:

```
SQL> DECLARE
  2      V_EMPNO EMP.EMPNO%TYPE;
  3
  4      E_EMP_NOT_FOUND EXCEPTION;
  5
  6  BEGIN
  7      V_EMPNO := &EMPNO;
  8
  9      DELETE FROM EMP
 10      WHERE EMPNO = V_EMPNO;
 11
 12      IF SQL%ROWCOUNT = 0 THEN
 13          RAISE E_EMP_NOT_FOUND;
 14      END IF;
 15
 16      COMMIT;
 17
 18      DBMS_OUTPUT.PUT_LINE(
 19          'Employee ' || V_EMPNO || ' deleted successfully.'
 20      );
 21
 22  EXCEPTION
 23
 24      WHEN E_EMP_NOT_FOUND THEN
 25          DBMS_OUTPUT.PUT_LINE(
 26              'Employee ' || V_EMPNO || ' not found.'
 27          );
 28
 29      WHEN OTHERS THEN
 30          DBMS_OUTPUT.PUT_LINE(
 31              'Other Error: ' || SQLERRM
 32          );
 33
 34  END;
 35  /
Enter value for empno: 7521
old   7:     V_EMPNO := &EMPNO;
new   7:     V_EMPNO := 7521;
Employee 7521 deleted successfully.
```

4. Create a stock table with columns PNO, PNAME, RATE AND TR_QTY.

Insert some records in this table.

Create a PL/SQL block that takes PNO, TR_TYPE and TR_QTY from the Stock Table.

If PNO is found and TR_TYPE is 'R' then update the TR_QTY with old TR_QTY plus new TR_QTY.

If PNO exists and TR_TYPE is 'I' then update the TR_QTY with old TR_QTY minus new TR_QTY.

Handle the user-defined exceptions using USER DEFINED EXCEPTION HANDLERS.

Query:

For Creating Table:

```
CREATE TABLE STOCK ( PNO NUMBER(4) PRIMARY KEY,
   PNAME VARCHAR2(30),
   RATE NUMBER(10,2),
   TR_QTY NUMBER(10));
```

For Inserting values into the table:

```
INSERT INTO STOCK VALUES (101, 'PEN', 10, 100);
INSERT INTO STOCK VALUES (102, 'PENCIL', 5, 200);
INSERT INTO STOCK VALUES (103, 'BOOK', 50, 150);
INSERT INTO STOCK VALUES (104, 'BAG', 500, 50);
INSERT INTO STOCK VALUES (105, 'SCALE', 20, 100);
COMMIT;
```

Checking the table:

```
SELECT * FROM STOCK;
```

Program:

```
SET SERVEROUTPUT ON;
DECLARE
   V_PNO     STOCK.PNO%TYPE;
   V_TR_TYPE CHAR(1);
   V_TR_QTY  STOCK.TR_QTY%TYPE;
   V_OLD_QTY STOCK.TR_QTY%TYPE;
   E_PNO_NOT_FOUND EXCEPTION;
   E_INVALID_TYPE  EXCEPTION;
```

```
E_INSUFFICIENT_QTY EXCEPTION;
BEGIN
   V_PNO := &PNO;
   V_TR_TYPE := UPPER('&TR_TYPE');
   V_TR_QTY := &TR_QTY;
 BEGIN
      SELECT TR_QTY
      INTO V_OLD_QTY
      FROM STOCK
      WHERE PNO = V_PNO;
  EXCEPTION
      WHEN NO_DATA_FOUND THEN
         RAISE E_PNO_NOT_FOUND;
   END;
  IF V_TR_TYPE = 'R' THEN
      UPDATE STOCK
      SET TR_QTY = TR_QTY + V_TR_QTY
      WHERE PNO = V_PNO;
    DBMS_OUTPUT.PUT_LINE('Stock received. Quantity increased.' );
  ELSIF V_TR_TYPE = 'I' THEN
     IF V_TR_QTY > V_OLD_QTY THEN
         RAISE E_INSUFFICIENT_QTY;
      END IF;
     UPDATE STOCK
      SET TR_QTY = TR_QTY - V_TR_QTY
      WHERE PNO = V_PNO;
       DBMS_OUTPUT.PUT_LINE('Stock issued. Quantity decreased.');
  ELSE
      RAISE E_INVALID_TYPE;
 END IF;
COMMIT;
EXCEPTION
  WHEN E_PNO_NOT_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Product number does not exist.' );
```

```
WHEN E_INVALID_TYPE THEN
    DBMS_OUTPUT.PUT_LINE('Invalid Transaction Type. Use R or I.' );
WHEN E_INSUFFICIENT_QTY THEN
    DBMS_OUTPUT.PUT_LINE('Insufficient Stock.' );
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Other Error: ' || SQLERRM);
END;
/
```

Output:

```
SQL> CREATE TABLE STOCK
  2  (
  3      PNO NUMBER(4) PRIMARY KEY,
  4      PNAME VARCHAR2(30),
  5      RATE NUMBER(10,2),
  6      TR_QTY NUMBER(10)
  7  );

Table created.

SQL> INSERT INTO STOCK VALUES (101, 'PEN', 10, 100);

1 row created.

SQL> INSERT INTO STOCK VALUES (102, 'PENCIL', 5, 200);

1 row created.

SQL> INSERT INTO STOCK VALUES (103, 'BOOK', 50, 150);

1 row created.

SQL> INSERT INTO STOCK VALUES (104, 'BAG', 500, 50);

1 row created.

SQL> INSERT INTO STOCK VALUES (105, 'SCALE', 20, 100);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

```
SQL> SELECT * FROM STOCK;

       PNO PNAME                          RATE     TR_QTY
---------- ------------------------------ ---------- ----------
       101 PEN                             10        100
       102 PENCIL                           5        200
       103 BOOK                            50        150
       104 BAG                            500         50
       105 SCALE                           20        100
```

```
SQL> DECLARE
  2      V_PNO     STOCK.PNO%TYPE;
  3      V_TR_TYPE CHAR(1);
  4      V_TR_QTY  STOCK.TR_QTY%TYPE;
  5
  6      V_OLD_QTY STOCK.TR_QTY%TYPE;
  7
  8      E_PNO_NOT_FOUND EXCEPTION;
  9      E_INVALID_TYPE  EXCEPTION;
 10      E_INSUFFICIENT_QTY EXCEPTION;
 11
 12  BEGIN
 13      V_PNO := &PNO;
 14      V_TR_TYPE := UPPER('&TR_TYPE');
 15      V_TR_QTY := &TR_QTY;
 16
 17      BEGIN
 18          SELECT TR_QTY
 19          INTO V_OLD_QTY
 20          FROM STOCK
 21          WHERE PNO = V_PNO;
 22
 23      EXCEPTION
 24          WHEN NO_DATA_FOUND THEN
 25              RAISE E_PNO_NOT_FOUND;
 26      END;
 27
 28      IF V_TR_TYPE = 'R' THEN
 29
 30          UPDATE STOCK
 31          SET TR_QTY = TR_QTY + V_TR_QTY
 32          WHERE PNO = V_PNO;
 33
 34          DBMS_OUTPUT.PUT_LINE(
 35              'Stock received. Quantity increased.'
 36          );
 37
 38      ELSIF V_TR_TYPE = 'I' THEN
 39
 40          IF V_TR_QTY > V_OLD_QTY THEN
```

```
 41             RAISE E_INSUFFICIENT_QTY;
 42         END IF;
 43
 44         UPDATE STOCK
 45         SET TR_QTY = TR_QTY - V_TR_QTY
 46         WHERE PNO = V_PNO;
 47
 48         DBMS_OUTPUT.PUT_LINE(
 49             'Stock issued. Quantity decreased.'
 50         );
 51
 52     ELSE
 53
 54         RAISE E_INVALID_TYPE;
 55
 56     END IF;
 57
 58     COMMIT;
 59
 60 EXCEPTION
 61
 62     WHEN E_PNO_NOT_FOUND THEN
 63         DBMS_OUTPUT.PUT_LINE(
 64             'Product number does not exist.'
 65         );
 66
 67     WHEN E_INVALID_TYPE THEN
 68         DBMS_OUTPUT.PUT_LINE(
 69             'Invalid Transaction Type. Use R or I.'
 70         );
 71
 72     WHEN E_INSUFFICIENT_QTY THEN
 73         DBMS_OUTPUT.PUT_LINE(
 74             'Insufficient Stock.'
 75         );
 76
 77     WHEN OTHERS THEN
 78         DBMS_OUTPUT.PUT_LINE(
 79             'Other Error: ' || SQLERRM
 80         );
 81
 82 END;
 83 /
Enter value for pno: 101
old  13:     V_PNO := &PNO;
new  13:     V_PNO := 101;
Enter value for tr_type: R
old  14:     V_TR_TYPE := UPPER('&TR_TYPE');
new  14:     V_TR_TYPE := UPPER('R');
Enter value for tr_qty: 50
old  15:     V_TR_QTY := &TR_QTY;
new  15:     V_TR_QTY := 50;
Stock received. Quantity increased.

PL/SQL procedure successfully completed.
```